

决策树

QwanxingxuA

2026 年 8 月 18 日

前记

关于决策树的相关内容，李航老师是写的非常规范的，本文并没有按照李航老师的结构写，更多地是记录笔者的想法。

笔者的学习顺序是乱的，涉及的内容也是杂乱无序的。如今，笔者不再认为自己是完全的初学者，所以笔者虽然也是第一次系统学习决策树，但是奇怪的想法就是会很多，所以写的内容可能与李航老师的差异很大，但是笔者确实是完全依照李航老师的书学习的，所以建议读者先阅读李航老师决策树那一章再来看本文。

本文可能存在错误，尤其是很多想法。权威的算法参考李航老师的书，当然笔者也不可能改变权威的算法，只是关于算法提出很多想法。

1 决策树

决策树的 ID3 算法、C4.5 算法、CART 算法在算法的流程上是被经常提到的，也不算困难。本文只简略提及算法流程，更多地关注决策树算法的数学逻辑以及模型、决策、算法的机器学习思想。

决策树可以看作是多层模型的连接，这点与神经网络类似。每一层可以说就是一个条件概率模型，以树的方式连接构成一个“扇形网络”模型，并且可观的是决策树的剪枝策略可以有类似于神经网络交叉熵损失函数的作用，只是可能很难具有反向传播这样方便的特性。

当然，可以更深地想到循环神经网络，笔者想过分类样本分布或者说信息会随着时间发生变化，分类也可能出现上下文关系，加之本身就有神经网络

络的特点，能不能也构建类似循环神经网络的循环决策树。当然笔者只是构想了一下，这其中可不可行尚未可知。

回归决策树，先给出前提数学定义：

样本空间 $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$ ，输入空间 $\mathcal{X}$ ，输出空间 $\mathcal{Y} = \{c_1, c_2, \dots, c_K\}$ 表示 K 分类任务。

定义单层条件概率 $P_A(Y | X)$ ：基于特征 A 分类的条件概率分布，假设使用某种策略 F 。则条件概率模型为：

$$\arg \min_{P_i(Y|X)} -F(P_i(Y | X)) \quad i = 1, 2, \dots, m$$

以上即从 m 个特征中选择最优的条件概率分布。当然这是从单层来看，实际上决策树的每一层之间存在联系，前面使用过的特征在后面不会考虑，所以上述定义不是严谨的。

下面定义决策树模型：假设决策树基于特征将原始样本分成 $|T|$ 个集合，则条件概率模型

$$P(y | x) = c_i I(x \in R_i), \quad i = 1, 2, \dots, |T|$$

其中， $I(\cdot)$ 是可式函数，这里即表示输入 x 在第 i 个集合； c_i 表示该集合分类到 y 的概率。

注：上述模型只取到概率大小，对于分类任务最终取概率值最大即可。更重要的是，这样定义还能与回归统一。笔者最初接触决策树的时候也认为其是分类模型，但实际上决策树也是回归模型，这点与神经网络又很类似了。实际上这应该是必然的，因为分类任务输出可以说就是回归任务输出的离散形式或者其实二者是可以通过映射相互转化的。

2 单层策略

本节基于单层模型与整体决策树的角度探讨机器学习的策略思想，简略探讨对应的算法以及探讨有意思的极大似然估计。李航老师多次提到极大似然估计，换作笔者以前在学校上课的时候或者说在笔者看最大熵那一章前，也完全是没觉得有什么，只是一带而过。实际上，这句话大有文章。

2.1 极大似然估计与最大熵

回顾最大熵模型，极大似然估计实际上是极大极小化问题中极大化问题的等价问题，可以直接说是最大熵问题的决策。通过最大熵模型构建了神经网络中大名鼎鼎的“softmax with crossentropyloss”层。迁移到上述关于构建决策树网络中，发现决策树每一层都要有自己的策略，这实际上是比较麻烦的，神经网络则不需要，笔者猜想这可能是目前没有实现决策树神经网络的重要原因。

当然更重要的是决策树也是基于观测数据得来的，基于已有信息获取模型分布。最大熵模型的熵是条件熵，但是决策树并不是直接最大化条件熵，而是考虑条件熵与经验熵的信息增益。在逻辑斯蒂回归模型中，往往都是仅仅考虑条件熵，但是在决策树里面也考虑经验熵。

无论怎样，把上述的策略思想沿用到单层策略上。回顾最大熵模型：

经验熵

$$H(P) = - \sum_{\mathbf{x}, y} \tilde{P}(\mathbf{x}, y) \log P(\mathbf{x}, y)$$

条件熵

$$H(P) = - \sum_{\mathbf{x}, y} \tilde{P}(\mathbf{x}, y) P(y | \mathbf{x}) \log P(y | \mathbf{x})$$

条件概率模型

$$P(y | \mathbf{x}) = \frac{\exp(\sum_{j=1}^m w_j f_j(\mathbf{x}, y))}{Z(\mathbf{x})}$$

其中

$$Z(\mathbf{x}) = \sum_y \exp(\sum_{j=1}^m w_j f_j(\mathbf{x}, y))$$

决策是极大似然估计。

2.2 信息增益

在决策树单层模型中，模型的表达形式有所变化，即从单一样本变成了集合或者说是单元。

基于特征 A 将样本 D 分为 n 个集合 $D_1, D_2, \dots, D_n$ ，记 C_k 表示第 k 类样本的个数， C_{ik} 表示 D_i 集合中 k 类样本的个数。则

经验熵

$$\begin{aligned}
 H(P) &= - \sum_{\mathbf{x}, y} \tilde{P}(\mathbf{x}, y) \log P(\mathbf{x}, y) \\
 &= - \sum_{k=1}^K \sum_{i=1}^n \frac{|C_{ik}|}{|D|} \log \frac{|C_{ik}|}{|D|} \\
 &= \sum_{k=1}^K \frac{|C_k|}{|D|} \log \frac{|C_k|}{|D|} \\
 &= H(D)
 \end{aligned}$$

条件熵

$$\begin{aligned}
 H(P) &= - \sum_{\mathbf{x}, y} \tilde{P}(\mathbf{x}, y) P(y | \mathbf{x}) \log P(y | \mathbf{x}) \\
 &= \sum_{k=1}^K \sum_{i=1}^n \frac{1}{|D_i|} \sum_{\mathbf{x} \in D_i} \frac{|C_{ik}|}{|D_i|} \log \frac{|C_{ik}|}{|D_i|} \\
 &= H(D | A)
 \end{aligned}$$

通过上面两个表达式可以发现，经验熵与特征 A 无关，条件熵与 A 有关。

决策树单层模型通过最大化信息增益选取特征 A 。

$$g(D, A) = H(D) - H(D, A)$$

2.3 决策树单层模型与最大熵模型对比，极大似然法

到这里可以发现最大熵模型与决策树在策略上出现了不同。最大熵原理是在已知信息约束下使得未知部分尽量“等可能”，其实就是让模型分布受未知信息影响减少，使得最大贴合数据分布，这实际上就是因为最大熵的极限情况下是均匀分布。决策树为什么不也使用这样极大似然的思想呢？笔者觉得还是因为这是单层模型，需要服务于总体决策树，服务于树的稳定性，所以考虑信息增益，通过局部的信息稳定性获取整体的信息稳定，这也是启发式思想。

这里会有一个很有意思的问题，既然是考虑稳定性强或者说信息不确定性低，为什么不直接极小化条件熵呢，这样还最稳定？实际上是因为它们

就是等价的（建议读者自己证明，当然使用信息增益肯定是更方便后续处理的）：

$$\max_A g(D, A) \Leftrightarrow \min_A H(D, A)$$

所以决策树和最大熵实际上是完全相反的路子了。但是无论极大极小化，因为最优化问题是可以相互转化的，最终都可以得到等价的极大似然估计。这也是李航老师为什么说“ID3 相当于用极大似然法进行概率模型的选择”。

以上便是 ID3 算法了，C4.5 算法对策略做了点变动，即最大化信息增益比：

$$g_R(D, A) = \frac{g(D, A)}{H_A(D)}$$

其中， $H_A(D)$ 是样本关于特征 A 的熵，假设特征 A 的取值为 $\{a_1, a_2, \dots, a_m\}$ ，则：

$$H_A(D) = - \sum_{i=1}^m \frac{|D_i|}{|D|} \log \frac{|D_i|}{|D|}$$

3 决策树策略：决策树剪枝

决策树模型整体的策略是决策树剪枝，只是不同的是，这个策略是针对泛化能力的。决策树的泛化能力和神经网络也很类似：层数很大的时候、模型很复杂的时候容易过拟合；在抑制过拟合的策略上也很类似：剪枝有点类似于 Dropout，只是决策树由于是扇形的，断了后面都会断，但是神经网络是网状的；决策树也使用正则化抑制，只是决策树是抑制叶节点数量，神经网络更多是对参数的抑制，但是何尝又不能说决策树的叶子节点也是参数。

假设决策树有 $|T|$ 个叶子节点， N_t 表示其中第 t 个叶子节点的样本数量。定义损失函数：

$$C_\alpha = \sum_{t=1}^{|T|} N_t H_t(T) + \alpha |T|$$

其中， α 是惩罚因子。

$$H_t(T) = - \sum_{k=1}^K \frac{|C_{tk}|}{N_t} \log \frac{|C_{tk}|}{N_t}$$

$$C_\alpha = - \sum_{t=1}^{|T|} \sum_{k=1}^K |C_{tk}| \frac{|C_{tk}|}{N_t} + \alpha |T|$$

通过选取子树，取上面最小的子树为最终决策树。这个过程基于某种方法可以通过动态规划实现，因为树之间是具有最优子问题性质和无后效性。

4 CART 算法

CART 算法实际上是构造二叉树。

4.1 回归问题

回归问题的输出空间 $\mathcal{Y}$ 是连续的。假设决策树将原始数据划分为 m 个集合 $\{R_1, R_2, \dots, R_M\}$ ，定义模型：

$$f(\mathbf{x}) = \sum_{i=1}^M c_i I(\mathbf{x} \in R_i)$$

使用平方误差作为损失函数。

CART 算法是构建二叉树，定义 (j, s) 表示以第 j 个特征划分出两个集合：

$$R_1 = \{\mathbf{x} \mid x^{(j)} \leq s\}, \quad R_2 = \{\mathbf{x} \mid x^{(j)} > s\}$$

于是，损失函数

$$\begin{aligned} L(j, s, c_1, c_2) &= \sum_{x_i \in R_1} (y_i - f(x_i))^2 + \sum_{x_i \in R_2} (y_i - f(x_i))^2 \\ &= \sum_{x_i \in R_1} (y_i - c_1)^2 + \sum_{x_i \in R_2} (y_i - c_2)^2 \end{aligned}$$

注意，这里损失函数是针对 c_1, c_2 而言的， (j, s) 是选择的决策，故而可以写成一个整体优化问题：

$$\min_{j, s} \min_{c_1, c_2} \sum_{x_i \in R_1} (y_i - c_1)^2 + \sum_{x_i \in R_2} (y_i - c_2)^2$$

这个函数很简单，求导即可。

4.2 分类问题

CART 算法分类问题使用基尼指数：

$$Gini(D) = 1 - \sum_{k=1}^K \frac{|C_k|}{|D|}^2$$

基于特征 A ，将样本分为 2 个集合

$$Gini(D, A) = \frac{|D_1|}{|D|}Gini(D_1) + \frac{|D_2|}{|D|}Gini(D_2)$$

CART 分类算法与 ID3 算法类似，只是取基尼指数最小，当然这其实也一样，因为前面说过取条件熵最小也是等价的。